SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: OPERATING SYSTEMS COURSE CODE: CSA04

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance(BL3-BL4)

Assessment Tool Weightage: 30%

QUESTIONS: 10 **Total Marks:50**

Annexure A

Scenario based Questions

Code of Conduct:

*I **A. Mounish / 192472273** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

Question 1 — CPU Scheduling in a Real-Time Ticket Booking System

A railway ticket booking system experiences heavy load during festival seasons. The system handles three categories of processes:

- P1 – Payment gateway transactions (High priority)
- P2 – Seat availability checks (Medium priority) • P3 – Ticket confirmation and printing (Low priority) • Burst times vary from 2 ms to 20 ms.

Recently, customers complained that ticket confirmations are taking too long. **Tasks**

1. Identify the most suitable scheduling algorithm (FCFS, SJF, Priority, RR) for this scenario.
2. Justify *why* your chosen method improves CPU utilization and reduces customer wait time.
3. Provide a sample execution order for the above categories.
4. Discuss one drawback of this scheduling method in real-time systems.

(10 Marks)

Question 2 — Deadlock in a Banking Transaction Server

A bank server handles the following operations simultaneously:

- T1: Transfers money between accounts
- T2: Updates passbook entries
- T3: Generates monthly statements

Due to simultaneous locking of Account Database (A), Customer Info Database (B), and Transaction Logs (C), the system enters a state where:

- T1 holds A and requests B
- T2 holds B and requests C
- T3 holds C and requests A

Tasks

1. Draw the resource request situation (text-based RAG).
2. Identify whether a deadlock has occurred and justify your answer.
3. Recommend *one specific technique* (avoidance/prevention/recovery) and explain how it fixes the issue.

4. Suggest a redesign for the locking order to avoid future deadlocks.

(10 Marks)

Question 3 — Mobile OS App Switching Lag

A mobile operating system is designed to support smooth app switching.

But users report that:

- Opening WhatsApp delays the background music player.
- Switching between social media apps feels laggy.
- CPU shows high idle times despite multiple running apps.

The OS currently uses Round Robin (quantum = 20 ms) with 20 background apps active.

Tasks

1. Analyze why the CPU remains idle despite many active processes.
2. Determine whether the time quantum is helping or hurting performance.
3. Recommend a better scheduling method or quantum value, with justification.
4. Explain how the change will reduce switching lag and improve system throughput.

(10 Marks) Question 4 — Deadlock Risk in an Operating System Update

During a Windows OS update, several modules run in parallel:

- M1: Copy update files
- M2: Install drivers
- M3: Update registry
- M4: Validate system integrity Resources involved:
- R1 = Disk I/O
- R2 = Kernel lock
- R3 = Configuration lock Recently logs show:
- M1 holds R1, waiting for R2
- M2 holds R2, waiting for R3
- M3 holds R3, waiting for R1

Tasks

1. Identify whether this sequence can lead to a deadlock.
2. If yes, explain the exact circular wait condition.
3. Propose *Banker's algorithm style* resource allocation to avoid risk.

4. Suggest how OS designers can prioritize modules to improve CPU utilization during update.

(10 Marks)

Question 5 — Cloud Server CPU Utilization Analysis A cloud hosting provider notices:

- 80% of the time, VM processes wait for network or disk
- Only 20% of the time is spent on CPU
- Each server runs 7 VMs Using the formula:

where p = I/O wait probability $U = 1 - p^n$

Tasks

1. Calculate CPU utilization.
2. Interpret whether the server is underutilized or optimally loaded.
3. Suggest how increasing or decreasing number of VMs affects throughput.
4. Propose two OS-level strategies for improving CPU utilization in this cloud environment.

(10 Marks) Question 6 – CPU Scheduling Scenario

Scenario:

A large-scale CPU scheduling environment in organization 1 experiences performance and reliability issues during peak usage. Multiple processes/resources interact simultaneously, causing delays, contention, and reduced throughput.

1. Analyze the root cause of the observed behavior.
2. Determine the most appropriate Operating System technique/algorithm to address the issue.
3. Justify your recommendation with respect to CPU utilization, throughput, response time, or fairness.
4. Design an improved system configuration or execution sequence for the given scenario.
5. Evaluate one limitation or trade-off of your proposed solution. **(10 Marks)**

Question 7:

****Scenario:****

A leading international e-commerce company operates across 70 countries and processes nearly 10 million customer requests every day. The system consists of multiple activities such as user authentication, product search, shopping cart updates, payment processing, inventory management, and shipment tracking. During festive sales, the response time of the application increases significantly due to the large number of simultaneously executing tasks. The Chief Technology Officer (CTO) has formed a task force to investigate how the operating system manages these activities as processes and how process state transitions affect overall system performance. The team is also interested in understanding the role of context switching and Process Control Blocks (PCB) in maintaining efficient execution.

****Questions:****

1. Analyze the various processes involved in the given scenario.
2. Differentiate between a program and a process with suitable examples from the case study.
3. Explain how Process Control Blocks (PCB) are utilized by the operating system.
4. Illustrate the different process states that a payment transaction may undergo.
5. Evaluate the impact of context switching during peak sale periods.
6. Predict the consequences of excessive process creation in the system.
7. Design an efficient process hierarchy for the organization.
8. Justify the need for multiprogramming in this environment.
9. Recommend strategies to improve process management and scalability.
10. Develop a process lifecycle model suitable for the e-commerce platform.

Question 8: Process Scheduling ****Scenario:**

A smart hospital has implemented an automated healthcare management system to handle patient registration, diagnostic services, medical record access, pharmacy management, and emergency response operations. During emergency situations, doctors have reported delays in retrieving patient records because CPU resources are being shared among multiple services. The hospital administration has requested the IT team to investigate different CPU

scheduling policies and recommend a suitable scheduling strategy that prioritizes emergency services without starving routine tasks.

****Questions:****

1. Identify the scheduling challenges present in the hospital management system.
2. Compare FCFS, SJF, Priority Scheduling, and Round Robin for this application.
3. Analyze the impact of scheduling policies on emergency patient care.
4. Determine the most appropriate scheduling algorithm for the given scenario.
5. Evaluate the possibility of starvation and suggest remedies.
6. Recommend a suitable time quantum if Round Robin is selected.
7. Predict system performance during a mass casualty incident.
8. Justify your choice of scheduling algorithm using performance metrics.
9. Propose modifications to improve fairness and responsiveness.
10. Design a scheduling framework for the hospital's future expansion plans.

Question 9: Inter-Process Communication and Cooperating Processes **Scenario:**

A smart city traffic management system integrates traffic signals, CCTV cameras, weather stations, public transportation services, and emergency vehicle tracking systems. These independent components continuously exchange information to optimize traffic flow and reduce congestion. However, communication failures between processes have occasionally resulted in delayed traffic signal updates and emergency response times. The city administration seeks an improved communication model to ensure uninterrupted coordination among all subsystems.

****Questions:****

1. Identify the cooperating processes involved in the smart city ecosystem.
2. Analyze the communication requirements among these processes.
3. Compare shared memory and message passing techniques for this environment.
4. Recommend the most suitable IPC mechanism and justify your choice.
5. Evaluate the impact of communication failures on public safety.

6. Propose a fault-tolerant IPC architecture for the city.
7. Predict potential challenges as the city expands its infrastructure.
8. Suggest methods to improve synchronization among cooperating processes.
9. Design a secure communication framework for the system.
10. Develop a scalability plan for future smart city implementations.

Question 10: Threads and Multithreading Models **Scenario:**

A global media company streams live sporting events to millions of viewers worldwide. To improve user experience, the application uses separate threads for video decoding, audio synchronization, subtitle generation, buffering, and advertisement insertion. During high-demand events such as the FIFA World Cup final, users occasionally experience video freezing and synchronization issues. The software architects have been asked to investigate whether the current multithreading model is appropriate for handling massive workloads.

****Questions:****

1. Identify the threads involved in the streaming application.
2. Compare user-level and kernel-level threads for this environment.
3. Analyze the suitability of One-to-One, Many-to-One, and Many-to-Many models.
4. Evaluate the consequences of thread blocking during live broadcasts.
5. Recommend an appropriate multithreading model with justification.
6. Predict system behavior under a tenfold increase in user traffic.
7. Suggest techniques to improve thread synchronization.
8. Design a thread management strategy for uninterrupted streaming.
9. Evaluate the role of multithreading in enhancing application performance.
10. Develop a monitoring framework for detecting thread-related issues.

Question 1 – CPU Scheduling in a Real-Time Ticket Booking System

1. Most suitable scheduling algorithm

Priority Scheduling (Preemptive)

2. Justification

- Payment gateway transactions (P1) are executed first because they are critical.
- Seat availability checks (P2) execute next.
- Ticket confirmation (P3) executes after higher-priority tasks.
- This reduces waiting time and improves CPU utilization during heavy booking traffic.

3. Sample execution order

Process	Priority	Execution Order
P1 – Payment Gateway	High	1st
P2 – Seat Availability	Medium	2nd
P3 – Ticket Confirmation	Low	3rd

Execution Sequence:
P1 → P2 → P3

4. Drawback

- Low-priority processes may suffer from **starvation** if high-priority jobs continuously arrive.

Question 2 – Deadlock in a Banking Transaction Server

1. Text-based Resource Allocation Graph (RAG)

T1 holds A → requests B

T2 holds B → requests C

T3 holds C → requests A

A → T1

T1 → B

B → T2

T2 → C

C → T3

T3 → A

2. Has a deadlock occurred?

Yes.

Justification:
There is a circular wait:

- T1 waits for B.
- T2 waits for C.
- T3 waits for A.
- None can continue.

3. Recommended Technique

Deadlock Prevention

Explanation:
Assign a fixed resource allocation order (A → B → C). Every transaction requests resources only in this order, preventing circular wait.

4. Redesign of Locking Order

All transactions should request resources as:

Account Database (A) → Customer Database (B) → Transaction Log (C)

This removes circular waiting.

Question 3 – Mobile OS App Switching Lag

1. Why is CPU idle despite many active processes?

- Many apps are waiting for I/O operations.
- Frequent context switching wastes CPU time.
- Processes remain in waiting state.

2. Is the 20 ms time quantum helping?

No.

A 20 ms quantum is too small for many background apps, causing excessive context switching and slower execution.

3. Better scheduling recommendation

Round Robin with 50 ms time quantum

Justification

- Reduces context switches.
- Gives processes more CPU time.
- Improves throughput and responsiveness.

4. How does it reduce lag?

- Less switching overhead.
- Faster completion of processes.
- Better CPU utilization.
- Smooth app switching and improved system throughput.

Question 4 – Deadlock Risk in an Operating System Update

1. Can this sequence lead to a deadlock?

Yes.

2. Explain the circular wait condition

The modules are waiting for each other's resources:

- **M1** holds **R1 (Disk I/O)** and waits for **R2 (Kernel Lock)**.
- **M2** holds **R2 (Kernel Lock)** and waits for **R3 (Configuration Lock)**.
- **M3** holds **R3 (Configuration Lock)** and waits for **R1 (Disk I/O)**.

Circular Wait:

M1 → R2 → M2 → R3 → M3 → R1 → M1

Hence, a deadlock can occur.

3. Banker's Algorithm Style Resource Allocation

- Allocate resources only if the system remains in a **safe state**.
- Check available resources before granting any request.
- Delay unsafe requests until resources become available.
- This prevents the system from entering a deadlock.

4. Improve CPU Utilization

- Give higher priority to critical update modules.

- Execute modules in the order: **M1 → M2 → M3 → M4**
- Release resources immediately after use.
- This reduces waiting time and improves CPU utilization.

Question 5 – Cloud Server CPU Utilization Analysis

Given

- I/O wait probability (**p**) = **0.8**
- Number of VMs (**n**) = **7**

Formula:

$$U = 1 - p^n$$

1. Calculate CPU Utilization

$$U = 1 - (0.8)^7$$
$$U = 1 - 0.2097$$
$$U = 0.7903$$

CPU Utilization = 79.03% (≈79%)

2. Interpretation

- CPU utilization is about **79%**.
- The server is **reasonably utilized**, but there is still room for improvement because many VMs spend time waiting for I/O.

3. Effect of Increasing/Decreasing VMs

- **Increasing VMs:** Better CPU utilization and higher throughput (up to an optimal limit).
- **Too many VMs:** Increased overhead and context switching.
- **Decreasing VMs:** Lower throughput and CPU utilization.

4. Two OS-Level Strategies

1. Use efficient CPU scheduling (e.g., Multi-level Feedback Queue).
2. Reduce I/O waiting using asynchronous I/O and faster storage (SSD/Caching).

Question 6 – CPU Scheduling Scenario

1. Analyze the Root Cause

- Heavy CPU contention during peak usage.
- Too many processes competing for CPU.
- Frequent context switching.
- Poor scheduling reduces throughput.

2. Suitable OS Technique

Multi-Level Feedback Queue (MLFQ) Scheduling

3. Justification

- Improves CPU utilization.
- Reduces response time.
- Provides fairness among processes.
- Increases overall throughput.

4. Improved System Configuration

- High-priority interactive processes execute first.
- CPU-bound processes move to lower-priority queues.
- Short jobs complete quickly.
- Long jobs continue without starving other processes.

Execution Order:
Interactive → Short Jobs → Long CPU-bound Jobs

5. Limitation

- MLFQ is more complex to implement.
- Requires careful tuning of queue priorities and time quantum.

Question 7 – E-Commerce Process Management

1. Analyze the various processes involved

The operating system manages several processes:

- User Authentication
- Product Search
- Shopping Cart Updates
- Payment Processing
- Inventory Management
- Shipment Tracking

2. Differentiate between Program and Process

Program	Process
A stored set of instructions	A program currently executing
Passive	Active
Example: Shopping application	Example: Running payment process

3. Explain Process Control Block (PCB)

PCB stores information about each running process such as:

- Process ID (PID)
- Process State
- Program Counter
- CPU Registers
- Scheduling Information
- Memory Information

The OS uses the PCB to manage and schedule processes efficiently.

4. Process States of Payment Transaction

New → Ready → Running → Waiting → Ready → Running → Terminated

- **New:** Payment request created.

- **Ready:** Waiting for CPU.
- **Running:** Payment is processed.
- **Waiting:** Waiting for database/network.
- **Ready:** Returns after I/O.
- **Running:** Completes execution.
- **Terminated:** Payment finishes.

5. Impact of Context Switching

- Increases CPU overhead.
- Slows response time.
- Reduces throughput.
- Causes delays during festive sales.

6. Consequences of Excessive Process Creation

- High memory usage.
- More context switching.
- CPU overload.
- Lower system performance.

7. Efficient Process Hierarchy

Parent Process

├── Authentication

├── Product Search

├── Shopping Cart

├── Payment

├── Inventory

└── Shipment Tracking

8. Need for Multiprogramming

- Executes multiple processes simultaneously.
- Improves CPU utilization.
- Reduces waiting time.
- Increases throughput.

9. Strategies to Improve Process Management

- Use Priority Scheduling.
- Apply Load Balancing.
- Use Thread Pools.
- Optimize Context Switching.

10. Process Lifecycle Model

New → Ready → Running → Waiting → Ready → Running → Terminated

Question 8 – Process Scheduling in Smart Hospital

1. Scheduling Challenges

- Emergency tasks delayed.
- CPU shared among many services.
- Long waiting time.
- Reduced responsiveness.

2. Comparison of Scheduling Algorithms

Algorithm	Suitability
FCFS	Simple but high waiting time
SJF	Low average waiting time
Priority	Best for emergency services
Round Robin	Fair CPU sharing

3. Impact on Emergency Patient Care

Poor scheduling delays:

- Patient record retrieval.
- Emergency treatment.
- Diagnosis.
- Medical decisions.

4. Most Suitable Algorithm

Priority Scheduling

Emergency services receive the highest priority.

5. Starvation and Remedy

Problem: Routine tasks may starve.

Solution: Aging technique increases the priority of waiting processes.

6. Suitable Time Quantum (if RR is used)

40–50 ms

Provides good responsiveness with fewer context switches.

7. Performance During Mass Casualty Incident

- CPU utilization becomes very high.
- Emergency tasks receive immediate CPU.
- Non-critical tasks execute later.

8. Justification

Priority Scheduling provides:

- Fast response time.
- Better CPU utilization.
- Quick emergency service.
- Improved throughput.

9. Improve Fairness

- Use Aging.
- Dynamic Priority Scheduling.
- Load Balancing.
- Increase CPU resources.

10. Scheduling Framework

Emergency Services

↓

Critical Medical Services

↓

Patient Records

↓

Pharmacy

↓

Routine Tasks

Question 9 – Inter-Process Communication (IPC) in Smart City

1. Cooperating Processes

- Traffic Signals
- CCTV Cameras
- Weather Stations
- Public Transport
- Emergency Vehicle Tracking

2. Communication Requirements

- Fast data exchange.
- Real-time updates.
- Reliable synchronization.

- Continuous communication.

3. Shared Memory vs Message Passing

Shared Memory	Message Passing
Faster	More secure
Needs synchronization	Easier communication
Best for same system	Best for distributed systems

4. Recommended IPC Mechanism

Message Passing

Reason: Smart city components run on different systems and networks.

5. Impact of Communication Failure

- Traffic congestion.
- Signal delays.
- Delayed emergency response.
- Increased accident risk.

6. Fault-Tolerant IPC

- Backup communication channels.
- Message queues.
- Automatic retry.
- Redundant servers.

7. Challenges During Expansion

- Higher network traffic.
- Increased synchronization.
- More communication overhead.
- Scalability issues.

8. Improve Synchronization

- Semaphores.
- Mutex Locks.
- Time Synchronization.
- Shared Buffers.

9. Secure Communication Framework

- Encryption.
- Authentication.
- Secure Message Queue.
- Access Control.

10. Scalability Plan

- Cloud-based infrastructure.
- Distributed IPC.
- Load balancing.
- Microservices architecture.

Question 10 – Threads and Multithreading Models

1. Threads Involved

- Video Decoding
- Audio Synchronization
- Subtitle Generation
- Buffering
- Advertisement Insertion

2. User-Level vs Kernel-Level Threads

User-Level	Kernel-Level
Faster creation	Better CPU scheduling

User-Level	Kernel-Level
Managed by user library	Managed by OS
Blocking affects all threads	One blocked thread doesn't stop others

3. Compare Thread Models

Model	Suitability
Many-to-One	Poor
One-to-One	Best
Many-to-Many	Good

4. Consequences of Thread Blocking

- Video freezing.
- Audio delay.
- Buffer interruption.
- Poor user experience.

5. Recommended Model

One-to-One Multithreading

Each user thread maps to one kernel thread, allowing true parallel execution and preventing one blocked thread from stopping others.

6. Behaviour with 10× More Users

- Higher CPU usage.
- Increased memory usage.
- More network traffic.
- Performance remains stable with proper load balancing.

7. Improve Thread Synchronization

- Mutex.
- Semaphores.
- Condition Variables.
- Read-Write Locks.

8. Thread Management Strategy

- Thread Pool.
- Dynamic Load Balancing.
- Priority Threads.
- Automatic Resource Allocation.

9. Role of Multithreading

- Better CPU utilization.
- Faster response.
- Smooth streaming.
- Higher throughput.

10. Monitoring Framework

- Monitor CPU usage.
- Monitor thread states.
- Detect deadlocks.
- Log synchronization issues.
- Generate performance reports.

RUBRICS FOR CALCULATION

Scenario based assessment

Criteria	**Weightage (%)**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand